

claude-windows / df5d5fb2-534a-447a-8181-b222f94655f3

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\df5d5fb2-534a-447a-8181-b222f94655f3\subagents\workflows\wf_f6b96576-946\agent-ad2e3f58a7bcfabd9.jsonl`  
- SHA-256: `3e7c2cb9a794d8b43b22361023337224ea89c58a736490e28e9d13d11d420789`  
- Source modified: `2026-07-06T18:06:12+00:00`  
- Imported at: `2026-07-08T15:59:36+00:00`  
- project: `wf_f6b96576-946`  
- session_id: `df5d5fb2-534a-447a-8181-b222f94655f3`
```

Transcript

1. user (2026-07-06T18:05:02.125Z)

You are an ADVERSARIAL VERIFIER. Try HARD to REFUTE this finding by reading the ACTUAL code in C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer (diff at C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/df5d5fb2-534a-447a-8181-b222f94655f3/scratchpad/wf294_full.diff). Default to REFUTED unless you can concretely reproduce the defect by pointing at real code lines AS WRITTEN. If a guard/caller already handles it, or the claimed line/behavior doesn't exist, or it's out-of-scope speculation -> REFUTED. If genuinely unclear after reading -> UNCERTAIN.

FINDING (convention-tests lens): severity=low | tests/test_tracknet_hybrid.py:100
summary: Of the F1/F2 pre-detection paths converted from bare []/None to Failure, only the missing-API-key path is locked by a test; unknown-sport, unknown-provider, invalid/<=0 duration, and native-not-supported are all silently uncovered in both the tracknet and yolo suites.
detail: F1 converted five trajectory_hybrid pre-detection paths to Failure: unknown sport (KeyError -> trajectory_hybrid.py:198), missing key (line 225), unknown provider (KeyError -> line ~231), invalid/<=0 duration (line ~236), and _resolve_runner NotImplementedError / native-not-supported (line ~262). F2 converted the yolo equivalents in _resolve_provider (yolo_hybrid.py:80/98/106) and process_video invalid-duration (line 514). The tests only lock the missing-API-key conversion: test_tracknet_hybrid.py:100
(test_pipeline_no_api_key_is_failure) and test_yolo_hybrid.py:201 (test_yolo_hybrid_no_api_key).
Grepping both test files for invalid-duration (get_video_duration return 0/negative), unknown/bad provider, unknown/bad sport, and NotImplementedError/native-runner yields no matches ? none of those four converted paths is asserted to return a Failure anywhere. So a regression that reverts any of them to `return []` (trajectory) or `return None` (yolo _resolve_provider) would reintroduce exactly the wf294 silent-'success' class and still pass the full suite. The healthcheck and no-trajectory paths (the earlier-reviewed change) ARE covered (test_tracknet_hybrid.py:129/152), and the legit-zero-windows empty and F3 all-errored/partial cases are covered ? it is specifically the F1/F2 config/env/input aborts other than missing-key that are unlocked.
failure_scenario: Someone edits _resolve_provider/_resolve_runner or the duration guard and accidentally restores `return []` / `return None` on the invalid-duration or native-not-supported path. The worker then writes status='success' with 0 segments for a broken video / unavailable detector (the wf294 silent failure), and the entire test suite stays green because no test exercises that path.
suggested_fix: Add parametrized tracknet twin tests and a yolo test asserting isinstance(res, Failure) for: get_video_duration <= 0 (invalid duration), a bad sport name (sport_registry KeyError), an unknown provider (provider_registry KeyError), and detector_impl='native' on tracknet (_resolve_runner NotImplementedError). Assert the message substring and, for consistency with the chosen convention, is_recoverable.

Verify against the real code and return your verdict.

2. assistant (2026-07-06T18:05:08.434Z)

I'll verify this finding by examining the actual test files and the converted code paths.

3. user (2026-07-06T18:05:09.433Z)

```
1      """Tests for the TrackNetHybridSegmenter pipeline. The WSL/TensorFlow runner is
2      mocked out (TrackNetRunner), so these run anywhere ? they exercise Phase 1-4 glue:
3      healthcheck gating, candidate persistence, AI handoff, and DB population/linking."""
4
5      from unittest.mock import MagicMock, patch
6
7      from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
8      from backend.results import Failure
9
10
11     def _window(start=1.0, end=4.0):
12         return {
13             "start": start,
14             "end": end,
15             "active_frame_count": 30,
16             "peak_velocity": 0.4,
17             "mean_velocity": 0.2,
18             "track_id_count": 1,
19             "chunk_index": 0,
20         }
21
22
23     def _patched(fn):
24         """Stack the patches every pipeline test needs. fps/width probing is stubbed
25         so cv2 internals don't need mocking (covered separately).
26
27         Patches are applied innermost-first, and mock.patch injects the innermost
28         patch as the *first* positional arg ? so this order matches the signature
29         (mock_runner_cls, mock_probe, mock_dur, mock_extract, mock_provider_registry,
30         mock_env)."""
31         fn = patch("backend.pipeline.segmenters.tracknet_hybrid.TrackNetRunner")(fn)
32         fn = patch.object(
33             TrackNetHybridSegmenter, "_probe_fps_width", return_value=(30.0, 1280.0)
34         )(fn)
35         # Shared pipeline collaborators live in the trajectory_hybrid base module.
36         fn = patch("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration")(fn)
37         fn = patch("backend.pipeline.segmenters.trajectory_hybrid.extract_video_chunk")(fn)
38         fn = patch("backend.pipeline.segmenters.trajectory_hybrid.provider_registry")(fn)
39         fn = patch("os.environ.get")(fn)
40         return fn
41
42
43     def _make_runner(windows=None, csv="out.csv", healthy=True):
44         runner = MagicMock()
45         runner.healthcheck.return_value = (
46             healthy,
47             "TF 2.17 GPUs 1" if healthy else "no GPU",
48         )
49         runner.run_predict.return_value = csv
50         runner.parse_trajectory_csv.return_value = [MagicMock(visible=True)]
51         runner.trajectory_to_action_windows.return_value = (
52             windows if windows is not None else [_window()]
53         )
54         return runner
55
56
57     @_patched
58     def test_pipeline_happy_path(
59         mock_runner_cls,
60         mock_probe,
61         mock_dur,
```

```

62         mock_extract,
63         mock_provider_registry,
64         mock_env,
65     ):
66         mock_env.return_value = "dummy_key"
67         mock_dur.return_value = 30.0
68         mock_extract.return_value = True
69         mock_runner_cls.return_value = _make_runner()
70
71         provider = MagicMock()
72         provider.analyze_video.return_value = (
73             [{"start_time": 0.5, "end_time": 3.0, "shuttle_exchange_count": 4}],
74             {},
75         )
76         mock_provider_registry.get.return_value = provider
77
78         db = MagicMock()
79         db.add_candidate_segment.return_value = 55
80         db.add_play_segment.return_value = 200
81
82         seg = TrackNetHybridSegmenter(db, {"indexing": {}})
83         res = seg.process_video("v1", "clip.mp4")
84
85         assert len(res) == 1
86         assert res[0]["id"] == 200
87         # Candidate persisted under the tracknet detector name...
88         _, kwargs = db.add_candidate_segment.call_args
89         assert kwargs["detector"] == "tracknet_hybrid"
90         # ...and the confirmed segment linked back to it.
91         db.link_candidate_to_segment.assert_called_once_with(55, 200)
92
93
94     @_patched
95     def test_pipeline_no_api_key_is_failure(
96         mock_runner_cls,
97         mock_probe,
98         mock_dur,
99         mock_extract,
100         mock_provider_registry,
101         mock_env,
102     ):
103         """A missing API key (with skip_ai_handoff=False) is an unrunnable misconfig ? a
Failure, NOT a
104         0-rally []. This is a wf294-class silent-failure the worker must mark failed, not
"success"."""
105         mock_env.return_value = None
106         mock_dur.return_value = 30.0
107         mock_runner_cls.return_value = _make_runner()
108
109         seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
110         res = seg.process_video("v1", "clip.mp4")
111         assert isinstance(res, Failure) and res.is_recoverable
112         assert "API_KEY" in res.message
113
114
115     @_patched
116     def test_pipeline_healthcheck_failure_aborts(
117         mock_runner_cls,
118         mock_probe,
119         mock_dur,
120         mock_extract,
121         mock_provider_registry,
122         mock_env,
123     ):
124         mock_env.return_value = "dummy_key"

```

```

125     mock_dur.return_value = 30.0
126     mock_extract.return_value = True
127     mock_runner_cls.return_value = _make_runner(healthy=False)
128     mock_provider_registry.get.return_value = MagicMock()
129
130     seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
131     res = seg.process_video("v1", "clip.mp4")
132     # A detector-env failure is a Failure, NOT a 0-rally [] ? so the caller marks the
run failed
133     # instead of writing an empty "success".
134     assert isinstance(res, Failure)
135     assert "environment not ready" in res.message
136     # Should bail before ever running inference.
137     mock_runner_cls.return_value.run_predict.assert_not_called()
138
139
140     @_patched
141     def test_pipeline_no_trajectory_aborts(
142         mock_runner_cls,
143         mock_probe,
144         mock_dur,
145         mock_extract,
146         mock_provider_registry,
147         mock_env,
148     ):
149         mock_env.return_value = "dummy_key"
150         mock_dur.return_value = 30.0
151         mock_runner_cls.return_value = _make_runner(csv=None) # predict produced nothing
152         mock_provider_registry.get.return_value = MagicMock()
153
154         seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
155         res = seg.process_video("v1", "clip.mp4")
156         # A detector timeout/abort (run_predict yielded no trajectory) is a Failure, NOT a
0-rally [] ?
157         # this is exactly the wf294 silent-failure that the caller must now mark
failed/retryable.
158         assert isinstance(res, Failure)
159         assert "no trajectory" in res.message and res.is_recoverable
160
161
162     @_patched
163     def test_pipeline_zero_windows_is_legit_empty_not_failure(
164         mock_runner_cls,
165         mock_probe,
166         mock_dur,
167         mock_extract,
168         mock_provider_registry,
169         mock_env,
170     ):
171         """The other side of the wf294 distinction: the detector RAN fine (a trajectory was
produced +
172         parsed) but yielded no candidate rally windows. That is a legitimately empty match ?
a bare []
173         (0-rally "success"), NOT a Failure."""
174         mock_env.return_value = "dummy_key"
175         mock_dur.return_value = 30.0
176         mock_extract.return_value = True
177         mock_runner_cls.return_value = _make_runner(windows=[]) # trajectory ok, zero
windows
178         mock_provider_registry.get.return_value = MagicMock()
179
180         seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
181         res = seg.process_video("v1", "clip.mp4")
182         assert res == [] and not isinstance(res, Failure)
183         # The detector DID run (a genuine empty, not an abort).

```

```

184         mock_runner_cls.return_value.run_predict.assert_called_once()
185
186
187     @_patched
188     def test_pipeline_store_candidates_disabled(
189         mock_runner_cls,
190         mock_probe,
191         mock_dur,
192         mock_extract,
193         mock_provider_registry,
194         mock_env,
195     ):
196         mock_env.return_value = "dummy_key"
197         mock_dur.return_value = 30.0
198         mock_extract.return_value = True
199         mock_runner_cls.return_value = _make_runner()
200
201         provider = MagicMock()
202         provider.analyze_video.return_value = ([{"start_time": 0.5, "end_time": 3.0}], {})
203         mock_provider_registry.get.return_value = provider
204
205         db = MagicMock()
206         db.add_play_segment.return_value = 200
207
208         seg = TrackNetHybridSegmenter(db, {"indexing": {"store_yolo_candidates": False}})
209         seg.process_video("v1", "clip.mp4")
210
211         db.add_candidate_segment.assert_not_called()
212         db.link_candidate_to_segment.assert_not_called()
213
214
215     @_patched
216     def test_pipeline_drops_unparseable_segments(
217         mock_runner_cls,
218         mock_probe,
219         mock_dur,
220         mock_extract,
221         mock_provider_registry,
222         mock_env,
223     ):
224         mock_env.return_value = "dummy_key"
225         mock_dur.return_value = 30.0
226         mock_extract.return_value = True
227         mock_runner_cls.return_value = _make_runner()
228
229         provider = MagicMock()
230         provider.analyze_video.return_value = (
231             [{"start_time": 0.5, "end_time": 3.0}, {"start_time": "N/A", "end_time": "??"}],
232             {},
233         )
234         mock_provider_registry.get.return_value = provider
235
236         db = MagicMock()
237         db.add_candidate_segment.return_value = 55
238         db.add_play_segment.return_value = 200
239
240         seg = TrackNetHybridSegmenter(db, {"indexing": {}})
241         res = seg.process_video("v1", "clip.mp4")
242         assert len(res) == 1 # the "N/A" segment is dropped
243
244
245     def test_probe_fps_width_fallbacks():
246         with patch(
247             "backend.pipeline.segmenters.trajectory_hybrid.cv2.VideoCapture"
248         ) as mock_cap:

```

```

249         bad = MagicMock()
250         bad.isOpened.return_value = False
251         bad.get.return_value = 0.0
252         mock_cap.return_value = bad
253         fps, width = TrackNetHybridSegmenter._probe_fps_width("missing.mp4")
254         assert fps == 30.0 and width == 1280.0
255
256
257     @_patched
258     def test_skip_ai_handoff_emits_candidates_as_rallies(
259         mock_runner_cls,
260         mock_probe,
261         mock_dur,
262         mock_extract,
263         mock_provider_registry,
264         mock_env,
265     ):
266         """Fully-local mode: candidate windows become rallies with NO provider/API key/clip
upload."""
267         mock_env.return_value = (
268             None # deliberately no API key ? fully-local must not need one
269         )
270         mock_dur.return_value = 30.0
271         windows = [_window(1.0, 4.0), _window(10.0, 13.0)]
272         mock_runner_cls.return_value = _make_runner(windows=windows)
273
274         db = MagicMock()
275         db.add_candidate_segment.side_effect = [101, 102]
276         db.add_play_segment.side_effect = [201, 202]
277
278         seg = TrackNetHybridSegmenter(db, {"indexing": {"skip_ai_handoff": True}})
279         res = seg.process_video("v1", "clip.mp4")
280
281         # Both candidate windows became rallies, verbatim, with no AI involvement.
282         assert len(res) == 2
283         mock_provider_registry.get.assert_not_called() # no provider built
284         mock_extract.assert_not_called() # no handoff clip extraction/upload
285         assert (res[0]["start_time"], res[0]["end_time"]) == (1.0, 4.0)
286         assert (res[1]["start_time"], res[1]["end_time"]) == (10.0, 13.0)
287         # Linked back to the persisted candidates.
288         db.link_candidate_to_segment.assert_any_call(101, 201)
289         # Local detector tag + confidence marker (metadata is the 6th positional arg).
290         meta = db.add_play_segment.call_args_list[0].args[5]
291         assert meta["detector"] == "tracknet-hybrid-local-noai"
292         assert meta["shot_count_confidence"] == "LocalNoAI"
293
294
295     def test_segementer_identity():
296         seg = TrackNetHybridSegmenter(MagicMock(), {})
297         assert seg.name == "tracknet_hybrid"
298         assert "TrackNet" in seg.description
299

```

4. user (2026-07-06T18:05:11.763Z)

```

1      diff --git a/backend/pipeline/segmenters/trajectory_hybrid.py
b/backend/pipeline/segmenters/trajectory_hybrid.py
2      index 503f6d8..7987278 100644
3      --- a/backend/pipeline/segmenters/trajectory_hybrid.py
4      +++ b/backend/pipeline/segmenters/trajectory_hybrid.py
5      @@ -22,7 +22,7 @@ import concurrent.futures
6      import logging
7      import os
8      import time
9      -from typing import Any, Dict, List, Optional, Tuple

```

```

10     +from typing import Any, Dict, List, Optional, Tuple, Union
11
12     import cv2
13
14     @@ -30,6 +30,9 @@ from backend.config.models import IndexingConfig
15     from backend.pipeline.detectors.base import DetectorRunner
16     from backend.pipeline.segmenters.base import BasePlaySegmenter
17     from backend.providers import provider_registry
18     +
19     +# Standardized Success|Failure contract: a detector Failure is NOT a 0-rally result.
20     +from backend.results import Failure
21     from backend.sports import sport_registry
22     from backend.utils.paths import output_base
23     from backend.utils.video import extract_video_chunk, get_video_duration
24     @@ -174,10 +177,14 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
25         video_path: str,
26         player_pool: Optional[List[str]] = None,
27         max_frames: Optional[int] = None,
28     - ) -> List[Dict[str, Any]]:
29     -     # override-ignore: the ABC declares the Result contract (Success|Failure)
30     -     # but every live segmenter still returns a bare list ? the documented
31     -     # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
32     + ) -> "Union[List[Dict[str, Any]], Failure]":
33     +     # Result contract (the ABC declares Success|Failure). Anything that means the
run could
34     +     # NOT actually complete ? an unrunnable config (unknown sport/provider, missing
API key),
35     +     # an unreadable video, an unavailable detector (native-not-supported), the env
healthcheck
36     +     # not ready, or run_predict timing out / aborting with no trajectory ? returns
a ``Failure``
37     +     # so it is NOT confused with a genuine 0-rally video. A LEGITIMATE empty match
(a trajectory
38     +     # was produced but no candidate/AI-confirmed rallies) still returns a bare
``[]``. Every
39     +     # caller (worker.cmd_infer, main.py, orchestration) branches on
isinstance(result, Failure).
40     label = self.display_label
41     # --- Sport profile + AI provider wiring (identical across segmenters) ---
42     sport_name = self.config.sport
43     @@ -185,7 +192,8 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
44         sport_profile = sport_registry.get(sport_name)
45         except KeyError as e:
46             logger.error(str(e))
47     -     return []
48     +     # Unrunnable config, not a 0-rally match (mirrors the gemini segmenter).
49     +     return Failure(str(e))
50
51     idx_cfg = self.config.indexing
52     skip_ai = bool(idx_cfg.skip_ai_handoff)
53     @@ -205,24 +213,31 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
54         if not api_key:
55             from backend.pipeline.segmenters._keyhint import api_key_hint
56
57     -     logger.error(
58     +     msg = (
59         f"{api_key_env_var} environment variable is not set! "
60         f"To run the {provider_name} handoff, set your API key. "
61         + api_key_hint(api_key_env_var)
62     )
63     -     return []
64     +     logger.error(msg)
65     +     # A misconfig (unrunnable), NOT a 0-rally match ? surface as Failure so
the run is
66     +     # marked failed instead of a silent empty "success" (mirrors gemini;

```

```

the worker's
67      +          # own 0-segment diagnostic names this missing-key case as a top cause).
68      +          return Failure(msg, is_recoverable=True)
69      try:
70          provider = provider_registry.get(
71              provider_name, api_key=api_key, model_name=model_name
72          )
73      except KeyError as e:
74          logger.error(str(e))
75      -          return []
76      +          return Failure(str(e))
77
78      video_duration = get_video_duration(video_path)
79      if video_duration <= 0:
80          logger.error("Invalid video duration.")
81      -          return []
82      +          # An unreadable/broken video, not a 0-rally match.
83      +          return Failure(
84      +              message=f"{label}: invalid/unreadable video duration
({video_duration}).",
85      +          )
86      fps, frame_width = self._probe_fps_width(video_path)
87
88      # --- Runner config + windowing thresholds ---
89      @@ -233,9 +248,10 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
90          runner, runner_params = self._resolve_runner(idx_cfg)
91      except NotImplementedError as e:
92          # e.g. detector_impl="native" for a family without a native runner
(tracknet, or
93      -          # fusion with a tracknet substrate). Refuse cleanly (no rallies) instead of
crashing.
94      +          # fusion with a tracknet substrate). An unavailable detector is a failure,
not a
95      +          # 0-rally match ? refuse cleanly as a Failure instead of crashing or a
false success.
96          logger.error("%s: %s", label, e)
97      -          return []
98      +          return Failure(message=f"{label}: {e}")
99
100     velocity_thresh = getattr(idx_cfg, self.family).velocity_threshold
101     merge_gap = idx_cfg.dead_time_merge_gap
102     @@ -267,7 +283,12 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
103         ok, msg = runner.healthcheck()
104         if not ok:
105             logger.error("%s detector environment not ready: %s", label, msg)
106         -          return []
107         +          # A detector FAILURE, not a 0-rally video ? surface it so the caller marks
the run
108         +          # failed/retryable instead of writing an empty-but-"successful" result.
109         +          return Failure(
110         +              message=f"{label} detector environment not ready: {msg}",
111         +              is_recoverable=True,
112         +          )
113         logger.info("%s detector env OK: %s", label, msg)
114
115     # --- Phase 2: trajectory -> candidate rally windows ---
116     @@ -278,7 +299,17 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
117         csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
118         if not csv_path:
119             logger.error("%s produced no trajectory; aborting.", label)
120         -          return []
121         +          # The cold-start / timeout failure mode (e.g. WASB native inference timed
out after
122         +          # timeout_sec, or the detector env yielded nothing). This is a DETECTOR
failure ? NOT

```

```

123      +          # a legitimately empty match ? so return a Failure the worker marks as
failed/retryable
124      +          # rather than a silent 0-segment "success" (the wf294 incident).
125      +          return Failure(
126      +              message=(
127      +                  f"{label} detector produced no trajectory (timed out or aborted) ?
"
128      +                  "detector failure, not a 0-rally video."
129      +              ),
130      +              is_recoverable=True,
131      +          )
132
133      points = runner.parse_trajectory_csv(csv_path)
134      logger.info(
135      @@ -387,7 +418,11 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
136          len(handoff_indices),
137      )
138
139      -          def process_candidate_window(wi: int, window: Dict[str, Any]) ->
List[dict]:
140      +          def process_candidate_window(
141      +              wi: int, window: Dict[str, Any]
142      +          ) -> Tuple[List[dict], bool]:
143      +              """Returns (valid_segments, errored) ? errored=True when this window
yielded NO
144      +              verdict (clip extraction failed, or the provider returned
metrics['error'])."""
145      +              w_start = max(0, window["start"] - handoff_padding)
146      +              w_end = min(video_duration, window["end"] + handoff_padding)
147      +              w_dur = w_end - w_start
148      @@ -395,7 +430,7 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
149      +              if not extract_video_chunk(
150      +                  video_path, w_start, w_dur, w_path, reencode=True
151      +              ):
152      -                  return []
153      +                  return [], True # extraction failed ? no verdict for this window
154      +                  prompt = sport_profile.get_prompt(chunk_duration=w_dur)
155      +                  segments, metrics = provider.analyze_video(
156      +                      w_path, prompt, chunk_duration=w_dur, label=f"Handoff {wi + 1}"
157      @@ -403,7 +438,8 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
158      +                  # B2: a provider failure returns [] with metrics["error"] set ? that is
159      +                  # "we don't know", NOT "no rallies in this window". Surface it loudly
so a
160      +                  # partial run isn't mistaken for a complete one (mirrors the gemini
segmenter).
161      -                  if metrics.get("error"):
162      +                  errored = bool(metrics.get("error"))
163      +                  if errored:
164      +                      logger.warning(
165      +                          "[Handoff %d] provider error ? window %d skipped: %s",
166      +                          wi + 1,
167      @@ -427,7 +463,7 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
168      +                      os.remove(w_path)
169      +                      except OSError:
170      +                          pass
171      -                  return valid
172      +                  return valid, errored
173
174      +              with concurrent.futures.ThreadPoolExecutor(
175      +                  max_workers=ai_max_workers
176      @@ -438,8 +474,10 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
177      +              )
178      +                  for wi in handoff_indices
179      +              ]
180      -          for future in futures:

```

```

181     -         ai_segments.extend(future.result())
182     +         window_results = [future.result() for future in futures]
183     +         for valid, _errored in window_results:
184     +             ai_segments.extend(valid)
185     +         handoff_failed = sum(1 for _valid, errored in window_results if errored)
186     +         try:
187     +             os.rmdir(handoff_dir)
188     +         except OSError:
189     @@ -448,6 +486,18 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
190     +             "Phase 3 completed. AI confirmed %d semantic play segments.",
191     +             len(ai_segments),
192     +         )
193     +         # wf294 (all-errored handoff): if EVERY handed-off window failed to yield a
verdict
194     +         # (provider error / clip extraction failed), the run has zero information ?
a FAILURE,
195     +         # not a 0-rally match. A single window that RAN and returned no rallies is
a legit empty
196     +         # and keeps this from firing. (skip_ai has no provider, so it never reaches
here.)
197     +         if handoff_indices and handoff_failed == len(handoff_indices):
198     +             msg = (
199     +                 f"{label} AI handoff produced no verdict: all {handoff_failed}
candidate "
200     +                 "window(s) errored (provider error / clip extraction failed) ?
incomplete "
201     +                 "run, not a 0-rally match."
202     +             )
203     +             logger.error(msg)
204     +             return Failure(message=msg, is_recoverable=True)
205
206     +         # --- Phase 4: DB population + candidate linking ---
207     +         segments_data = []
208     diff --git a/backend/pipeline/segmenters/yolo_hybrid.py
b/backend/pipeline/segmenters/yolo_hybrid.py
209     index d9334f1..2e2e1fe 100644
210     --- a/backend/pipeline/segmenters/yolo_hybrid.py
211     +++ b/backend/pipeline/segmenters/yolo_hybrid.py
212     @@ -22,11 +22,14 @@ import concurrent.futures
213     import logging
214     import os
215     import time
216     -from typing import Any, Dict, List, Tuple
217     +from typing import Any, Dict, List, Tuple, Union
218
219     from backend.pipeline.detectors.person_detector import PersonDetector
220     from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
221     from backend.providers import provider_registry
222     +
223     +# Standardized Success|Failure contract: an unrunnable config is NOT a 0-rally result.
224     +from backend.results import Failure
225     from backend.sports import sport_registry
226     from backend.utils.paths import output_base
227     from backend.utils.video import extract_video_chunk, get_video_duration
228     @@ -63,9 +66,10 @@ class HybridYoloSegmenter(BasePlaySegmenter):
229     def _resolve_provider(self, video_path: str):
230     +        """Phase 0: resolve sport profile + AI provider (with API key).
231
232     -        Returns the configured provider instance, or ``None`` to signal that
233     -        ``process_video`` should early-return ``[]`` (bad sport, missing API key,
234     -        or unknown provider). Behaviour is identical to the inlined version.
235     +        Returns the configured provider instance, or a ``Failure`` (bad sport, missing
API key,
236     +        or unknown provider) that ``process_video`` propagates ? so an unrunnable
config is marked

```

```

237 +         FAILED with a reason instead of a silent 0-rally "success" (the wf294 loud-
failures bar;
238 +         mirrors the gemini/trajectory segmenters). A genuine "ran, found nothing" is
unaffected.
239         """
240         # Get sport profile config
241         sport_name = self.config.sport
242     @@ -73,7 +77,7 @@ class HybridYoloSegmenter(BasePlaySegmenter):
243         sport_registry.get(sport_name)
244     except KeyError as e:
245         logger.error(str(e))
246 -         return None
247 +         return Failure(str(e))
248
249         # Get AI provider and model config
250         provider_name = self.config.ai_provider
251     @@ -85,12 +89,13 @@ class HybridYoloSegmenter(BasePlaySegmenter):
252         if not api_key:
253             from backend.pipeline.segmenters._keyhint import api_key_hint
254
255 -         logger.error(
256 +         msg = (
257             f"{api_key_env_var} environment variable is not set! "
258             f"To run the {provider_name} segmenter, set your API key. "
259             + api_key_hint(api_key_env_var)
260         )
261 -         return None
262 +         logger.error(msg)
263 +         return Failure(msg, is_recoverable=True)
264
265         try:
266             provider = provider_registry.get(
267     @@ -98,7 +103,7 @@ class HybridYoloSegmenter(BasePlaySegmenter):
268         )
269     except KeyError as e:
270         logger.error(str(e))
271 -         return None
272 +         return Failure(str(e))
273
274         logger.info(
275             f"Initializing Hybrid Segmenter (torchvision + ByteTrack) for video:
{video_path}"
276     @@ -320,9 +325,13 @@ class HybridYoloSegmenter(BasePlaySegmenter):
277         chunk_duration: float,
278         handoff_padding: float,
279         ai_max_workers: int,
280 -    ) -> List[dict]:
281 +    ) -> "Union[List[dict], Failure]":
282         """Phase 3: AI Provider Handoff ? re-encode each padded candidate window and
ask
283 -         the provider for semantic boundaries, in parallel. Verbatim move."""
284 +         the provider for semantic boundaries, in parallel.
285 +
286 +         Returns the confirmed segments, or a ``Failure`` when EVERY candidate window
failed to
287 +         yield a verdict (provider error / clip extraction failed) ? an incomplete run
has zero
288 +         information and must NOT be recorded as a 0-rally "success" (the wf294
class)."""
289         gemini_segments = []
290         gemini_dir = os.path.join(
291             output_base(self.config), f"chunks_{provider_name}_handoff"
292     @@ -333,7 +342,11 @@ class HybridYoloSegmenter(BasePlaySegmenter):
293         f"Starting Phase 3: AI Provider ({provider_name}) Validation on
{len(all_candidate_windows)} candidate windows..."

```

```

294         )
295
296     -     def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
297     +     def process_candidate_window(
298     +         wi: int, window: Dict[str, Any]
299     +     ) -> Tuple[List[dict], bool]:
300     +         """Returns (valid_segments, errored) ? errored=True when this window
yielded NO verdict
301     +         (clip extraction failed, or the provider returned metrics['error'])."""
302     +         w_start = max(0, window["start"] - handoff_padding)
303     +         w_end = min(video_duration, window["end"] + handoff_padding)
304     +         w_dur = w_end - w_start
305     @@ -344,7 +357,7 @@ class HybridYoloSegmenter(BasePlaySegmenter):
306     +         video_path, w_start, w_dur, w_path, reencode=True
307     +     )
308     +     if not success:
309     -         return []
310     +         return [], True # extraction failed ? no verdict for this window
311
312     +     prompt = sport_profile.get_prompt(chunk_duration=w_dur)
313     +     segments, metrics = provider.analyze_video(
314     @@ -353,7 +366,8 @@ class HybridYoloSegmenter(BasePlaySegmenter):
315     +     # B2: a provider failure returns [] with metrics["error"] set ? that is
316     +     # "we don't know", NOT "no rallies in this window". Surface it loudly so a
317     +     # partial run isn't mistaken for a complete one (mirrors the gemini
segmenter).
318     -         if metrics.get("error"):
319     +         errored = bool(metrics.get("error"))
320     +         if errored:
321     +             logger.warning(
322     +                 "[Handoff %d] provider error ? window %d skipped: %s",
323     +                 wi + 1,
324     @@ -380,7 +394,7 @@ class HybridYoloSegmenter(BasePlaySegmenter):
325     +         except OSError:
326     +             pass
327
328     -         return valid_segments
329     +         return valid_segments, errored
330
331     +     with concurrent.futures.ThreadPoolExecutor(
332     +         max_workers=ai_max_workers
333     @@ -390,8 +404,10 @@ class HybridYoloSegmenter(BasePlaySegmenter):
334     +         for wi, w in enumerate(all_candidate_windows)
335     +     ]
336     +     # Iterate in submission order to preserve temporal ordering
337     -     for future in futures:
338     -         gemini_segments.extend(future.result())
339     +     window_results = [future.result() for future in futures]
340     +     for valid, _errored in window_results:
341     +         gemini_segments.extend(valid)
342     +     handoff_failed = sum(1 for _valid, errored in window_results if errored)
343
344     +     try:
345     +         os.rmdir(gemini_dir)
346     @@ -401,6 +417,17 @@ class HybridYoloSegmenter(BasePlaySegmenter):
347     +         logger.info(
348     +             f"Phase 3 completed. AI confirmed {len(gemini_segments)} semantic play
segments."
349     +         )
350     +         # wf294 (all-errored handoff): if EVERY candidate window failed to yield a
verdict
351     +         # (provider error / clip extraction failed), the run has zero information ? a
FAILURE, not
352     +         # a 0-rally match. A window that RAN and returned no rallies keeps this from
firing.

```

```

353 +         if all_candidate_windows and handoff_failed == len(all_candidate_windows):
354 +             msg = (
355 +                 f"yolo_hybrid AI handoff produced no verdict: all {handoff_failed}
candidate "
356 +                 "window(s) errored (provider error / clip extraction failed) ?
incomplete run, "
357 +                 "not a 0-rally match."
358 +             )
359 +             logger.error(msg)
360 +             return Failure(message=msg, is_recoverable=True)
361         return gemini_segments
362
363     def _populate_db(
364     @@ -456,19 +483,23 @@ class HybridYoloSegmenter(BasePlaySegmenter):
365
366         return segments_data
367
368 -     def process_video(
369 +     def process_video( # type: ignore[override]
370         self,
371         video_id: str,
372         video_path: str,
373         player_pool: List[str] = None,
374         max_frames: int = None,
375 -     ) -> List[Dict[str, Any]]:
376 +     ) -> "Union[List[Dict[str, Any]], Failure]":
377 +         # Result contract: an unrunnable config (bad sport/provider, missing key) or an
unreadable
378 +         # video returns a Failure so the run is marked failed ? NOT a silent 0-rally
"success"
379 +         # (the wf294 class). A genuine empty match still returns a bare []. Every
caller branches
380 +         # on isinstance(result, Failure).
381         self.person.lazy_load_model()
382
383 -         # Phase 0: resolve sport profile + AI provider (early-returns [] on any
failure).
384 +         # Phase 0: resolve sport profile + AI provider (returns a Failure on any config
failure).
385         provider = self._resolve_provider(video_path)
386         if provider is None:
387             return []
388 +         if isinstance(provider, Failure):
389 +             return provider
390
391         # Re-read the config values needed downstream (pure deterministic reads).
392         idx_cfg = self.config.indexing
393     @@ -479,7 +510,10 @@ class HybridYoloSegmenter(BasePlaySegmenter):
394         video_duration = get_video_duration(video_path)
395         if video_duration <= 0:
396             logger.error("Invalid video duration.")
397             return []
398 +         # An unreadable/broken video, not a 0-rally match.
399 +         return Failure(
400 +             message=f"yolo_hybrid: invalid/unreadable video duration
({video_duration})."
401 +         )
402
403         chunk_duration = idx_cfg.chunk_duration
404         chunk_overlap = idx_cfg.chunk_overlap
405     @@ -537,8 +571,8 @@ class HybridYoloSegmenter(BasePlaySegmenter):
406         except Exception:
407             pass
408
409 -         # Phase 3: AI Provider Handoff

```

```

410 -         gemini_segments = self._run_ai_handoff(
411 +         # Phase 3: AI Provider Handoff (returns a Failure when every window errored ?
wf294).
412 +         handoff = self._run_ai_handoff(
413             video_id,
414             video_path,
415             video_duration,
416 @@ -551,9 +585,11 @@ class HybridYoloSegmenter(BasePlaySegmenter):
417             handoff_padding,
418             ai_max_workers,
419         )
420 +         if isinstance(handoff, Failure):
421 +             return handoff
422
423         # Phase 4: DB Population
424 -         return self._populate_db(video_id, gemini_segments, model_name)
425 +         return self._populate_db(video_id, handoff, model_name)
426
427
428     # Register the segmenter
429 diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
430 index 461d7ca..88c50f6 100644
431 --- a/backend/worker/__main__.py
432 +++ b/backend/worker/__main__.py
433 @@ -114,6 +114,42 @@ def _write_report_json(
434     )
435
436
437 +def _serialize_and_push_outputs(
438 +    scratch: str,
439 +    out_uri: str,
440 +    video_id: str,
441 +    segments: List[dict],
442 +    status: str,
443 +    storage_cfg: dict,
444 +    video_uri: str = "",
445 +) -> List[str]:
446 +    """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and
push both to
447 +    ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.
448 +
449 +    Shared by the SUCCESS path and the failure path (``_fail``) so a detector
timeout/abort leaves a
450 +    ``status="failed"`` report.json in the bucket ? the control plane's source of truth
451 +    (``orchestration.cached_process_result`` / ``probe_process_cache`` treat any
non-``success``
452 +    report as "no usable result" ? retry, and the restart-recovery poll waits for
report.json to
453 +    EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``.
454 +    out_local = os.path.join(scratch, "outputs", video_id)
455 +    os.makedirs(out_local, exist_ok=True)
456 +    _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
457 +    _write_report_json(
458 +        video_id,
459 +        segments,
460 +        status,
461 +        os.path.join(out_local, "report.json"),
462 +        video_uri=video_uri,
463 +    )
464 +    out_prefix = out_uri.rstrip("/")
465 +    pushed: List[str] = []
466 +    for fn in sorted(os.listdir(out_local)):
467 +        uri = f"{out_prefix}/outputs/{video_id}/{fn}"
468 +        storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
469 +        pushed.append(uri)

```

```

470     +     return pushed
471     +
472     +
473     def _run_startup_checks(config: Any) -> bool:
474         """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
475         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
476         @@ -237,9 +273,26 @@ def cmd_infer(args: argparse.Namespace) -> int:
477             )
478
479         def _fail(rc: int) -> int:
480             -         # M6: on a worker-side FAILURE, write the done-marker (the REAL rc, 0 segments)
so a remote
481             -         # dispatcher's bucket-poll terminates FAST + accurately instead of hanging
until its poll
482             -         # budget expires. DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
483             +         # A worker-side FAILURE (validation, unknown segmenter, or a detector
timeout/abort the
484             +         # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
Write + push
485             +         # a status="failed" report.json/intervals.csv (0 segments) so the control
plane's bucket
486             +         # truth (submit-cache probe, restart-recovery) and the alpha-user result
surface see an
487             +         # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
must never
488             +         # mask the original failure. THEN the M6 done-marker (real rc + status) so a
remote
489             +         # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
expires.
490             +         try:
491             +             _serialize_and_push_outputs(
492             +                 scratch,
493             +                 args.out_uri,
494             +                 video_id,
495             +                 [],
496             +                 "failed",
497             +                 storage_cfg,
498             +                 str(getattr(args, "video_uri", "") or ""),
499             +             )
500             +         except Exception as e: # never mask the real failure on an artifact-push
hiccup
501             +             logger.warning("[worker] could not push failure artifacts: %s", e)
502             +             # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
503             +             if getattr(args, "done_uri", None):
504             +                 _write_done_marker(
505             +                     args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
506             @@ -313,24 +366,16 @@ def cmd_infer(args: argparse.Namespace) -> int:
507                 )
508
509             # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
510             -         out_local = os.path.join(scratch, "outputs", video_id)
511             -         os.makedirs(out_local, exist_ok=True)
512             -         _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
513             -         _write_report_json(
514             +         pushed = _serialize_and_push_outputs(
515             +             scratch,
516             +             args.out_uri,
517             +             video_id,
518             +             segments,
519             +             status,
520             -             os.path.join(out_local, "report.json"),
521             -             video_uri=str(getattr(args, "video_uri", "") or ""),

```

```

522 +         storage_cfg,
523 +         str(getattr(args, "video_uri", "") or ""),
524     )
525
526 -     out_prefix = args.out_uri.rstrip("/")
527 -     pushed = []
528 -     for fn in sorted(os.listdir(out_local)):
529 -         uri = f"{out_prefix}/outputs/{video_id}/{fn}"
530 -         storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
531 -         pushed.append(uri)
532 -
533     print(
534         f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):"
535     )
536     diff --git a/tests/test_tracknet_hybrid.py b/tests/test_tracknet_hybrid.py
537     index 64909e7..c157ed5 100644
538     --- a/tests/test_tracknet_hybrid.py
539     +++ b/tests/test_tracknet_hybrid.py
540     @@ -5,6 +5,7 @@ healthcheck gating, candidate persistence, AI handoff, and DB
population/linking
541     from unittest.mock import MagicMock, patch
542
543     from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
544     +from backend.results import Failure
545
546
547     def _window(start=1.0, end=4.0):
548     @@ -91,7 +92,7 @@ def test_pipeline_happy_path(
549
550
551     @_patched
552     -def test_pipeline_no_api_key_returns_empty(
553     +def test_pipeline_no_api_key_is_failure(
554         mock_runner_cls,
555         mock_probe,
556         mock_dur,
557     @@ -99,12 +100,16 @@ def test_pipeline_no_api_key_returns_empty(
558         mock_provider_registry,
559         mock_env,
560     ):
561     +     """A missing API key (with skip_ai_handoff=False) is an unrunnable misconfig ? a
Failure, NOT a
562     +     0-rally []. This is a wf294-class silent-failure the worker must mark failed, not
"success"."""
563         mock_env.return_value = None
564         mock_dur.return_value = 30.0
565         mock_runner_cls.return_value = _make_runner()
566
567         seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
568     -     assert seg.process_video("v1", "clip.mp4") == []
569     +     res = seg.process_video("v1", "clip.mp4")
570     +     assert isinstance(res, Failure) and res.is_recoverable
571     +     assert "API_KEY" in res.message
572
573
574     @_patched
575     @@ -124,7 +129,10 @@ def test_pipeline_healthcheck_failure_aborts(
576
577         seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
578         res = seg.process_video("v1", "clip.mp4")
579     -     assert res == []
580     +     # A detector-env failure is a Failure, NOT a 0-rally [] ? so the caller marks the
run failed
581     +     # instead of writing an empty "success".

```

```

582 +     assert isinstance(res, Failure)
583 +     assert "environment not ready" in res.message
584 +     # Should bail before ever running inference.
585 +     mock_runner_cls.return_value.run_predict.assert_not_called()
586
587 @@ -144,7 +152,36 @@ def test_pipeline_no_trajectory_aborts(
588     mock_provider_registry.get.return_value = MagicMock()
589
590     seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
591 -     assert seg.process_video("v1", "clip.mp4") == []
592 +     res = seg.process_video("v1", "clip.mp4")
593 +     # A detector timeout/abort (run_predict yielded no trajectory) is a Failure, NOT a
0-rally [] ?
594 +     # this is exactly the wf294 silent-failure that the caller must now mark
failed/retryable.
595 +     assert isinstance(res, Failure)
596 +     assert "no trajectory" in res.message and res.is_recoverable
597 +
598 +
599 +@_patched
600 +def test_pipeline_zero_windows_is_legit_empty_not_failure(
601 +     mock_runner_cls,
602 +     mock_probe,
603 +     mock_dur,
604 +     mock_extract,
605 +     mock_provider_registry,
606 +     mock_env,
607 +):
608 +     """The other side of the wf294 distinction: the detector RAN fine (a trajectory was
produced +
609 +     parsed) but yielded no candidate rally windows. That is a legitimately empty match
? a bare []
610 +     (0-rally "success"), NOT a Failure."""
611 +     mock_env.return_value = "dummy_key"
612 +     mock_dur.return_value = 30.0
613 +     mock_extract.return_value = True
614 +     mock_runner_cls.return_value = _make_runner(windows=[]) # trajectory ok, zero
windows
615 +     mock_provider_registry.get.return_value = MagicMock()
616 +
617 +     seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
618 +     res = seg.process_video("v1", "clip.mp4")
619 +     assert res == [] and not isinstance(res, Failure)
620 +     # The detector DID run (a genuine empty, not an abort).
621 +     mock_runner_cls.return_value.run_predict.assert_called_once()
622
623
624     @_patched
625     diff --git a/tests/test_trajectory_hybrid_equivalence.py
b/tests/test_trajectory_hybrid_equivalence.py
626     index 0080f4d..0270e41 100644
627     --- a/tests/test_trajectory_hybrid_equivalence.py
628     +++ b/tests/test_trajectory_hybrid_equivalence.py
629     @@ -14,6 +14,7 @@ import pytest
630     from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
631     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
632     from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
633 +from backend.results import Failure
634
635     TWINS = [
636         (TrackNetHybridSegmenter, "tracknet", "tracknet_hybrid"),
637     @@ -111,8 +112,10 @@ def test_provider_offsets_applied_identically(cls, family,
detector):
638
639     @pytest.mark.parametrize("cls,family,detector", TWINS)

```

```

640     def test_provider_failure_is_surfaced_not_silent(cls, family, detector, caplog):
641     -     """B2/P17: a provider failure ([], metrics['error']) must be logged loudly, not
silently
642     -     treated as 'no rallies in this window'. The run still returns [] (no analysis
done)."""
643     +     """B2/P17 + F3 (wf294): a provider failure ([], metrics['error']) must be logged
loudly, not
644     +     silently treated as 'no rallies'. When it errors on the ONLY candidate window the
whole run has
645     +     zero verdict, so process_video now returns a Failure (was a bare [] that looked
like a 0-rally
646     +     match) ? the worker marks it failed/retryable instead of a silent empty
'success'."""
647         import logging
648
649         provider = MagicMock()
650     @@ -139,13 +142,45 @@ def test_provider_failure_is_surfaced_not_silent(cls, family,
detector, caplog):
651             logging.WARNING, logger="backend.pipeline.segmenters.trajectory_hybrid"
652             ):
653                 res = seg.process_video("v1", "clip.mp4")
654     -     assert res == [] # no analysis for the failed window
655     +     assert isinstance(res, Failure) and res.is_recoverable # all (1) windows errored
-> no verdict
656         assert any(
657             "provider error" in r.message and "generate failed" in r.message
658             for r in caplog.records
659         ), [r.message for r in caplog.records]
660
661
662     +@pytest.mark.parametrize("cls,family,detector", TWINS)
663     +def test_partial_handoff_error_is_a_success_not_a_failure(cls, family, detector):
664     +     """The boundary of F3: when SOME candidate windows error but at least one RAN (even
if it found
665     +     no rallies), the run is NOT a failure ? only an all-errored handoff is. Here two
windows are
666     +     handed off; the first returns a rally, the second errors -> a Success with the
first rally."""
667     +     provider = MagicMock()
668     +     provider.analyze_video.side_effect = [
669     +         ([{"start_time": 0.5, "end_time": 2.0}], {}), # window 0: a real verdict
670     +         ([], {"error": "generate failed after retries: 503"}), # window 1: errored
671     +     ]
672     +     db = MagicMock()
673     +     db.add_candidate_segment.side_effect = [55, 56]
674     +     db.add_play_segment.return_value = 200
675     +     two_windows = [_window(1.0, 4.0), _window(10.0, 13.0)]
676     +     with patch.object(
677     +         cls, "_build_runner", return_value=(_runner(two_windows), {"weights_path":
"w"}))
678     +     ), patch.object(cls, "_probe_fps_width", return_value=(30.0, 1280.0)), patch(
679     +         "backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
680     +         return_value=30.0,
681     +     ), patch(
682     +         "backend.pipeline.segmenters.trajectory_hybrid.extract_video_chunk",
683     +         return_value=True,
684     +     ), patch(
685     +         "backend.pipeline.segmenters.trajectory_hybrid.provider_registry"
686     +     ) as preg, patch("os.environ.get", return_value="dummy_key"):
687         preg.get.return_value = provider
688         seg = cls(db, {"indexing": {}})
689         res = seg.process_video("v1", "clip.mp4")
690         assert not isinstance(res, Failure)
691         assert len(res) == 1 and res[0]["start_time"] == 0.5
692     +

```

```

693 +
694 def test_base_is_not_registered():
695     from backend.pipeline.segmenters.base import segmenter_registry
696
697 diff --git a/tests/test_worker.py b/tests/test_worker.py
698 index 96edbee..8fb5e16 100644
699 --- a/tests/test_worker.py
700 +++ b/tests/test_worker.py
701 @@ -10,6 +10,7 @@ import json
702     import pytest
703
704     from backend.config import load_config
705 +from backend.results import Failure
706     from backend.worker import __main__ as wmain
707
708     # ----- pure helpers
709 @@ -191,7 +192,7 @@ def test_cmd_infer_local_roundtrip(tmp_path, mocked_pipeline):
710
711
712     class _EmptySeg:
713 -        """A segmenter that finds nothing ? the cold-start failure mode."""
714 +        """A segmenter that finds nothing ? a LEGITIMATE 0-rally match (a bare [])."""
715
716         def __init__(self, db, config):
717             pass
718 @@ -200,6 +201,20 @@ class _EmptySeg:
719             return []
720
721
722 +class _FailSeg:
723 +    """A segmenter whose DETECTOR failed/timed out ? returns a Failure (e.g. WASB
produced no
724 +    trajectory), which is NOT the same as a 0-rally match. This is the wf294 shape."""
725 +
726     def __init__(self, db, config):
727         pass
728
729     def process_video(self, video_id, path, max_frames=None):
730         return Failure(
731             message="WASB detector produced no trajectory (timed out or aborted)",
732             is_recoverable=True,
733         )
734
735
736     def test_setup_logging_levels():
737         import logging
738
739 @@ -246,12 +261,112 @@ def test_cmd_infer_zero_segments_is_logged_loudly(tmp_path,
monkeypatch, caplog)
740         assert "0 segments" in msgs and "vidZ" in msgs
741         assert "detector_impl=native" in msgs # surfaces the resolved detector
742         assert "native runner UNAVAILABLE" in msgs # points at the likely cause
743         assert (
744             json.loads((out / "outputs" / "vidZ" / "report.json").read_text())[
745                 "segment_count"
746             ] == 0
747         ) # A segmenter that returns a bare [] is a LEGITIMATE 0-rally match ? status stays
"success"
747 +    # (contrast test_cmd_infer_detector_failure_*, where a Failure surfaces as
status="failed").
748     report = json.loads((out / "outputs" / "vidZ" / "report.json").read_text())
749     assert report["segment_count"] == 0 and report["status"] == "success"
750
751
752 +def test_cmd_infer_detector_failure_marks_report_and_marker_failed(
753     tmp_path, monkeypatch

```

```

754     +):
755     +     """wf294 fix: a segmenter that returns a Failure (detector timeout/abort) must NOT
be written
756     +     as a 0-segment "success". cmd_infer returns rc 3 and writes report.json + the done-
marker with
757     +     status="failed" so the control plane / alpha-user surface can show an error /
retry."""
758     +     monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidX")
759     +     monkeypatch.setattr(
760     +         wmain.validator_registry,
761     +         "run_all_with_context",
762     +         lambda path, cfg: ([_OKResult()], {}),
763     +     )
764     +     monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FailSeg)
765     +     monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
766     +     video = tmp_path / "in.mp4"
767     +     video.write_bytes(b"FAKE")
768     +     out = tmp_path / "bucket"
769     +     done = tmp_path / "bucket" / "_dispatch" / "tok.done"
770     +
771     +     rc = wmain.main(
772     +         [
773     +             "infer",
774     +             "--video-uri",
775     +             str(video),
776     +             "--out-uri",
777     +             str(out),
778     +             "--scratch",
779     +             str(tmp_path / "s"),
780     +             "--segmenter",
781     +             "wasb_hybrid",
782     +             "--done-uri",
783     +             str(done),
784     +         ]
785     +     )
786     +     # A detector failure is rc 3 (distinct from validation's rc 2 and a clean rc 0).
787     +     assert rc == 3
788     +     # report.json is written as an explicit FAILURE, not a false empty "success".
789     +     report = json.loads((out / "outputs" / "vidX" / "report.json").read_text())
790     +     assert report["status"] == "failed" and report["segment_count"] == 0
791     +     assert report["segments"] == []
792     +     # ...and the completion marker the cloud dispatcher bucket-polls carries the same
verdict.
793     +     marker = json.loads(done.read_text())
794     +     assert marker["returncode"] == 3 and marker["status"] == "failed"
795     +     assert marker["segment_count"] == 0 and marker["video_id"] == "vidX"
796     +
797     +
798     + def test_cmd_infer_real_trajectory_segmenter_failure_flows_to_report(tmp_path,
monkeypatch):
799     +     """End-to-end: the ACTUAL WasbHybridSegmenter no-trajectory abort (the wf294 shape)
flows
800     +     through cmd_infer -> _fail(3) -> a status="failed" report.json + done-marker. Locks
the two
801     +     halves (segmenter Failure + worker report/marker) TOGETHER, not just via the
_FailSeg stub."""
802     +     from unittest.mock import MagicMock
803     +
804     +     from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
805     +
806     +     runner = MagicMock()
807     +     runner.healthcheck.return_value = (True, "env ok")
808     +     runner.run_predict.return_value = None # WASB timed out / aborted: no trajectory
produced
809     +

```

```

810 + # Force the real base process_video down to its no-trajectory abort without a
GPU/video.
811 + monkeypatch.setattr(WasbHybridSegmenter, "_resolve_runner", lambda self, cfg:
(runner, {}))
812 + monkeypatch.setattr(
813 +     WasbHybridSegmenter, "_probe_fps_width", lambda *a, **k: (30.0, 1280.0)
814 + )
815 + monkeypatch.setattr(
816 +     "backend.pipeline.segmenters.trajectory_hybrid.get_video_duration", lambda p:
30.0
817 + )
818 + monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidR")
819 + monkeypatch.setattr(
820 +     wmain.validator_registry, "run_all_with_context", lambda path, cfg:
([_OKResult()], {}))
821 + )
822 + monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name:
WasbHybridSegmenter)
823 + monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
824 +
825 + video = tmp_path / "in.mp4"
826 + video.write_bytes(b"FAKE")
827 + out = tmp_path / "bucket"
828 + done = tmp_path / "bucket" / "_dispatch" / "tok.done"
829 +
830 + rc = wmain.main(
831 +     [
832 +         "infer",
833 +         "--video-uri",
834 +         str(video),
835 +         "--out-uri",
836 +         str(out),
837 +         "--scratch",
838 +         str(tmp_path / "s"),
839 +         "--segmenter",
840 +         "wasb_hybrid",
841 +         "--done-uri",
842 +         str(done),
843 +         "--set",
844 +         "indexing.skip_ai_handoff=true", # no API key needed; isolate the detector
abort
845         ]
846 -     == 0
847 )
848 + assert rc == 3
849 + report = json.loads((out / "outputs" / "vidR" / "report.json").read_text())
850 + assert report["status"] == "failed" and report["segment_count"] == 0
851 + marker = json.loads(done.read_text())
852 + assert marker["returncode"] == 3 and marker["status"] == "failed"
853
854
855 def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
856     @@ -266,20 +381,30 @@ def test_cmd_infer_validation_failure_returns_2(tmp_path,
monkeypatch):
857         video.write_bytes(b"X")
858         cfg = tmp_path / "config.json"
859         cfg.write_text("{}")
860 +     out = tmp_path / "o"
861 +     done = tmp_path / "o" / "_dispatch" / "tok.done"
862         rc = wmain.main(
863             [
864                 "infer",
865                 "--video-uri",
866                 str(video),
867                 "--out-uri",

```

```

868     -         str(tmp_path / "o"),
869     +         str(out),
870     +         "--config",
871     +         str(cfg),
872     +         "--scratch",
873     +         str(tmp_path / "s"),
874     +         "--done-uri",
875     +         str(done),
876     +     ]
877     + )
878     + assert rc == 2
879     + # A validation failure is ALSO written as an explicit failure (not a missing
artifact / false
880     + # success): report.json status="failed" + the done-marker rc 2 / status="failed".
881     + report = json.loads((out / "outputs" / "vidF" / "report.json").read_text())
882     + assert report["status"] == "failed" and report["segment_count"] == 0
883     + marker = json.loads(done.read_text())
884     + assert marker["returncode"] == 2 and marker["status"] == "failed"
885
886
887     # --- cmd_train: pull labels -> synth trajectories -> manifest -> harness -> push
bundle ---
888     diff --git a/tests/test_yolo_hybrid.py b/tests/test_yolo_hybrid.py
889     index 38e78b2..09da6c2 100644
890     --- a/tests/test_yolo_hybrid.py
891     +++ b/tests/test_yolo_hybrid.py
892     @@ -11,6 +11,7 @@ import os
893     from unittest.mock import MagicMock, patch
894
895     from backend.pipeline.segmenters.yolo_hybrid import HybridYoloSegmenter
896     +from backend.results import Failure
897
898
899     def _make_cap_mock(num_frames=300):
900     @@ -110,8 +111,10 @@ def test_yolo_hybrid_segmenter(
901     def test_yolo_hybrid_provider_failure_is_surfaced_not_silent(
902     mock_env_get, mock_provider_registry, mock_extract, mock_get_dur, mock_cap, caplog
903     ):
904     -     """B2/P17: a provider failure ([], metrics['error']) must be logged loudly, not
silently
905     -     treated as 'no rallies'. The run still returns [] (no analysis done)."""
906     +     """B2/P17 + F3 (wf294): a provider failure ([], metrics['error']) must be logged
loudly, not
907     +     silently treated as 'no rallies'. When it errors on the ONLY candidate window the
whole run has
908     +     zero verdict, so process_video now returns a Failure (was a bare [] that looked
like a 0-rally
909     +     match) ? the worker marks it failed/retryable instead of a silent empty
'success'."""
910     import logging
911
912     mock_env_get.return_value = "dummy_api_key"
913     @@ -147,13 +150,49 @@ def test_yolo_hybrid_provider_failure_is_surfaced_not_silent(
914     logging.WARNING, logger="backend.pipeline.segmenters.yolo_hybrid"
915     ):
916     res = segmenter.process_video("v1", "dummy.mp4")
917     - assert res == [] # no analysis for the failed window
918     + assert isinstance(res, Failure) and res.is_recoverable # all (1) windows errored
-> no verdict
919     assert any(
920     "provider error" in r.message and "generate failed" in r.message
921     for r in caplog.records
922     ), [r.message for r in caplog.records]
923
924

```

```

925 +@patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
926 +@patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk")
927 +@patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
928 +@patch("os.environ.get")
929 +def test_yolo_hybrid_partial_handoff_error_is_a_success_not_a_failure(
930 +    mock_env_get, mock_provider_registry, mock_extract, mock_get_dur
931 +):
932 +    """The boundary of F3: SOME windows errored but at least one RAN, so the run is a
Success (the
933 +    first window's rally), NOT a Failure. Stage 2 is patched to hand off two windows
directly."""
934 +    mock_env_get.return_value = "dummy_api_key"
935 +    mock_get_dur.return_value = 30.0
936 +    mock_extract.return_value = True
937 +
938 +    mock_provider = MagicMock()
939 +    mock_provider.analyze_video.side_effect = [
940 +        ([{"start_time": 0.5, "end_time": 2.0}], {}), # window 0: a real verdict
941 +        ([], {"error": "generate failed after retries: 503"}), # window 1: errored
942 +    ]
943 +    mock_provider_registry.get.return_value = mock_provider
944 +
945 +    db_mock = MagicMock()
946 +    db_mock.add_play_segment.return_value = "yolo_segment_1"
947 +    two_windows = [{"start": 1.0, "end": 4.0}, {"start": 10.0, "end": 13.0}]
948 +
949 +    segmenter = HybridYoloSegmenter(db_mock, {"indexing": {"use_gpu": False}})
950 +    segmenter.person.model = MagicMock() # short-circuit lazy model load
951 +    with patch.object(
952 +        HybridYoloSegmenter, "_collect_candidate_windows", return_value=two_windows
953 +    ), patch.object(
954 +        HybridYoloSegmenter, "_persist_candidates", return_value=[None, None]
955 +    ):
956 +        res = segmenter.process_video("v1", "dummy.mp4")
957 +        assert not isinstance(res, Failure)
958 +        assert len(res) == 1 and res[0]["start_time"] == 0.5
959 +
960 +
961 +@patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
962 +@patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
963 +@patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk")
964 @@ -162,7 +201,8 @@ def test_yolo_hybrid_provider_failure_is_surfaced_not_silent(
965     def test_yolo_hybrid_no_api_key(
966         mock_env_get, mock_provider_registry, mock_extract, mock_get_dur, mock_cap
967     ):
968         - """Segmenter should return empty when GEMINI_API_KEY is missing."""
969         + """A missing API key is an unrunnable misconfig ? a Failure, NOT a 0-rally [] that
the worker
970         +    would write as a silent status="success" (the wf294 class; yolo_hybrid is the
default segmenter)."""
971         mock_env_get.return_value = None
972
973         db_mock = MagicMock()
974 @@ -172,7 +212,8 @@ def test_yolo_hybrid_no_api_key(
975         segmenter.person.model = MagicMock() # short-circuit lazy model load
976
977         res = segmenter.process_video("v1", "dummy.mp4")
978         - assert res == []
979         + assert isinstance(res, Failure) and res.is_recoverable
980         + assert "API_KEY" in res.message
981
982
983 +@patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
984

```

5. assistant (2026-07-06T18:05:17.909Z)

The finding claims that of the F1/F2 pre-detection paths converted to `Failure`, only the missing-API-key path has a test. Let me verify by checking the actual test files for coverage of the other paths.

6. user (2026-07-06T18:05:21.023Z)

```
14:from backend.results import Failure
59:@patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
61:@patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
64:    mock_env_get, mock_provider_registry, mock_extract, mock_get_dur, mock_cap
70:    # Mock provider returned play segment
71:    mock_provider = MagicMock()
72:    mock_provider.analyze_video.return_value = (
76:    mock_provider_registry.get.return_value = mock_provider
84:        "chunk_duration": 100,
90:        "min_action_window_duration": 1.0,
107:@patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
109:@patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
111:def test_yolo_hybrid_provider_failure_is_surfaced_not_silent(
112:    mock_env_get, mock_provider_registry, mock_extract, mock_get_dur, mock_cap, caplog
114:    """B2/P17 + F3 (wf294): a provider failure ([], metrics['error']) must be logged loudly,
not
116:    zero verdict, so process_video now returns a Failure (was a bare [] that looked like a
0-rally
124:    mock_provider = MagicMock()
125:    mock_provider.analyze_video.return_value = (
129:    mock_provider_registry.get.return_value = mock_provider
136:        "chunk_duration": 100,
141:        "min_action_window_duration": 1.0,
153:    assert isinstance(res, Failure) and res.is_recoverable # all (1) windows errored -> no
verdict
155:        "provider error" in r.message and "generate failed" in r.message
160:@patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
162:@patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
165:    mock_env_get, mock_provider_registry, mock_extract, mock_get_dur
168:    first window's rally), NOT a Failure. Stage 2 is patched to hand off two windows
directly."""
173:    mock_provider = MagicMock()
174:    mock_provider.analyze_video.side_effect = [
178:    mock_provider_registry.get.return_value = mock_provider
192:    assert not isinstance(res, Failure)
197:@patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
199:@patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
202:    mock_env_get, mock_provider_registry, mock_extract, mock_get_dur, mock_cap
204:    """A missing API key is an unrunnable misconfig ? a Failure, NOT a 0-rally [] that the
worker
215:    assert isinstance(res, Failure) and res.is_recoverable
220:@patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
222:@patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
225:    mock_env_get, mock_provider_registry, mock_extract, mock_get_dur, mock_cap
233:    mock_provider = MagicMock()
234:    mock_provider.analyze_video.return_value = (
241:    mock_provider_registry.get.return_value = mock_provider
249:        "chunk_duration": 100,
254:        "min_action_window_duration": 0.5,
272:    config = {"indexing": {"min_action_window_duration": 1.0}}
305:@patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
307:@patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
310:    mock_env_get, mock_provider_registry, mock_extract, mock_get_dur, mock_cap
317:    mock_provider = MagicMock()
318:    mock_provider.analyze_video.return_value = (
322:    mock_provider_registry.get.return_value = mock_provider
331:        "chunk_duration": 100,
```

```

336:         "min_action_window_duration": 1.0,
358:@patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
360:@patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
363:     mock_env_get, mock_provider_registry, mock_extract, mock_get_dur, mock_cap
370:     mock_provider = MagicMock()
371:     mock_provider.analyze_video.return_value = (
375:     mock_provider_registry.get.return_value = mock_provider
383:         "chunk_duration": 100,
388:         "min_action_window_duration": 1.0,
404:@patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
406:@patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
412:     mock_provider_registry,
417:     """M3: yolo's chunk-staging (`chunks_yolo`) + AI-handoff (`chunks_<provider>_handoff`)
dirs
422:     mock_provider = MagicMock()
423:     mock_provider.analyze_video.return_value = (
427:     mock_provider_registry.get.return_value = mock_provider
435:         "chunk_duration": 100,
440:         "min_action_window_duration": 1.0,

```

7. user (2026-07-06T18:05:22.711Z)

```

29:     (mock_runner_cls, mock_probe, mock_dur, mock_extract, mock_provider_registry,
36:     fn = patch("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration")(fn)
38:     fn = patch("backend.pipeline.segmenters.trajectory_hybrid.provider_registry")(fn)
63:     mock_provider_registry,
76:     mock_provider_registry.get.return_value = provider
100:     mock_provider_registry,
121:     mock_provider_registry,
128:     mock_provider_registry.get.return_value = MagicMock()
146:     mock_provider_registry,
152:     mock_provider_registry.get.return_value = MagicMock()
168:     mock_provider_registry,
178:     mock_provider_registry.get.return_value = MagicMock()
193:     mock_provider_registry,
203:     mock_provider_registry.get.return_value = provider
221:     mock_provider_registry,
234:     mock_provider_registry.get.return_value = provider
263:     mock_provider_registry,
283:     mock_provider_registry.get.assert_not_called() # no provider built

```

8. assistant (2026-07-06T18:05:26.275Z)

Now let me confirm the finding's claims about which paths are tested. The claim is that four converted paths (invalid duration, unknown provider, unknown sport, native-not-supported) are uncovered in both suites. Let me search across ALL test files to make sure there isn't coverage elsewhere (e.g., the equivalence test file, or a dedicated config-validation test).

9. user (2026-07-06T18:05:29.917Z)

```

tests\test_trajectory_hybrid_equivalence.py:54:         db.add_play_segment.return_value = 200
tests\test_trajectory_hybrid_equivalence.py:58:         ), patch.object(cls, "_probe_fps_width",
return_value=(30.0, 1280.0)), patch(
tests\test_trajectory_hybrid_equivalence.py:60:             return_value=30.0,
tests\test_trajectory_hybrid_equivalence.py:92:             (None, 3), # explicit null -> duration
estimate (2.5s/0.8 -> 3 floor)
tests\test_trajectory_hybrid_equivalence.py:130:         ), patch.object(cls, "_probe_fps_width",
return_value=(30.0, 1280.0)), patch(
tests\test_trajectory_hybrid_equivalence.py:132:             return_value=30.0,
tests\test_trajectory_hybrid_equivalence.py:164:         db.add_play_segment.return_value = 200
tests\test_trajectory_hybrid_equivalence.py:168:         ), patch.object(cls, "_probe_fps_width",
return_value=(30.0, 1280.0)), patch(
tests\test_trajectory_hybrid_equivalence.py:170:             return_value=30.0,

```

10. user (2026-07-06T18:05:31.365Z)

```
tests\test_api_cloud_dispatch.py:183:    assert "indexing.detector_impl=native" in joined
tests\test_automated_provider.py:65:    with pytest.raises(NotImplementedError):
tests\test_compute_backend.py:224:    assert "indexing.detector_impl=native" in joined
tests\test_config_deployment.py:129:    assert cfg.indexing.detector_impl == "native"
tests\test_config_deployment.py:140:    # detector_impl is intentionally left at the production
default (env picks WSL vs native).
tests\test_config_deployment.py:157:    "indexing": {"detector_impl": "native"}, #
override one preset knob...
tests\test_config_deployment.py:161:    assert cfg.indexing.detector_impl == "native" # file
wins
tests\test_config_deployment.py:288:    assert cfg.indexing.detector_impl == "auto" # default,
not "native"
tests\test_config_wasb.py:36:    # (unchanged Windows behaviour); the Linux-native runner
defaults to STREAMING (the perf win,
tests\test_config_wasb.py:54:    flags omitted). An explicit override reaches the native runner
cfg ? the seam the
tests\test_native_wasb_runner.py:34:    """The whole point of M3.5: the native runner must carry
NO WSL plumbing."""
tests\test_native_wasb_runner.py:370:    """Unmocked: the native runner stages the SAME
wasb_infer.py the WSL runner runs (the
tests\test_native_wasb_runner.py:461:# --- the detector_impl='native' seam (parallels the stub
seam) -----
tests\test_native_wasb_runner.py:462:def
test_resolve_runner_native_selects_native_runner_for_wasb(monkeypatch):
tests\test_native_wasb_runner.py:466:    runner, params = WasbHybridSegmenter(None,
{})._resolve_runner(
tests\test_native_wasb_runner.py:467:        {"detector_impl": "native"}
tests\test_native_wasb_runner.py:470:    assert params.get("detector_impl") == "native"
tests\test_native_wasb_runner.py:473:def
test_resolve_runner_native_env_overrides_config(monkeypatch):
tests\test_native_wasb_runner.py:477:    runner, _ = WasbHybridSegmenter(None,
{})._resolve_runner({"detector_impl": "auto"})
tests\test_native_wasb_runner.py:481:def
test_resolve_runner_native_supported_for_fusion_wasb(monkeypatch):
tests\test_native_wasb_runner.py:485:    runner, params = FusionSegmenter(None,
{})._resolve_runner(
tests\test_native_wasb_runner.py:486:        {"detector_impl": "native"}
tests\test_native_wasb_runner.py:492:def
test_resolve_runner_native_refused_for_fusion_tracknet(monkeypatch):
tests\test_native_wasb_runner.py:496:    with pytest.raises(NotImplementedError):
tests\test_native_wasb_runner.py:497:        FusionSegmenter(None, {})._resolve_runner(
tests\test_native_wasb_runner.py:498:            {"detector_impl": "native", "fusion":
{"substrate": "tracknet"}}
tests\test_native_wasb_runner.py:502:def
test_resolve_runner_native_refused_for_tracknet(monkeypatch):
tests\test_native_wasb_runner.py:506:    with pytest.raises(NotImplementedError):
tests\test_native_wasb_runner.py:507:        TrackNetHybridSegmenter(None,
{})._resolve_runner({"detector_impl": "native"})
tests\test_startup_checks.py:228:    cfg = _cfg(profile="cpu-mock", backend="local",
detector_impl="native")
tests\test_storage.py:99:    with pytest.raises(NotImplementedError):
tests\test_storage.py:535:    with pytest.raises(NotImplementedError):
tests\test_stub_runner.py:5:(model-agnostic) helpers, and the `_resolve_runner` hook swaps it in
via config/env
tests\test_stub_runner.py:99:def test_resolve_runner_default_delegates_to_build_runner(seg,
monkeypatch):
tests\test_stub_runner.py:101:    runner, params = seg._resolve_runner({})
tests\test_stub_runner.py:105:def test_resolve_runner_config_selects_stub(seg, monkeypatch):
tests\test_stub_runner.py:107:    runner, params = seg._resolve_runner({"detector_impl":
"stub"})
tests\test_stub_runner.py:112:def test_resolve_runner_env_overrides_config(seg, monkeypatch):
tests\test_stub_runner.py:115:    runner, _ = seg._resolve_runner({"detector_impl": "auto"})
tests\test_tracknet_runner.py:572:    """TrackNetRunner must be a DetectorRunner (for
```

```

polymorphism / future native runners)."""
tests\test_upload_gcs_streaming.py:11:- a backend whose open_writer raises NotImplementedError
falls back to the local flow
tests\test_upload_gcs_streaming.py:204:         raise NotImplementedError("this backend cannot
stream")
tests\test_worker.py:262:     assert "detector_impl=native" in msgs # surfaces the resolved
detector
tests\test_worker.py:263:     assert "native runner UNAVAILABLE" in msgs # points at the likely
cause
tests\test_worker.py:329:     monkeypatch.setattr(WasbHybridSegmenter, "_resolve_runner", lambda
self, cfg: (runner, {}))
tests\test_worker.py:860:     """detector_impl=native for a family with no native runner must
return rc 2 (clean), not
tests\test_worker.py:861:     propagate a raw NotImplementedError traceback out of the worker."""
tests\test_worker.py:893:     ) # graceful: _resolve_train_detector caught NotImplementedError,
returned None

```

11. assistant (2026-07-06T18:05:36.148Z)

This is important. The finding claims the "native-not-supported" path (`_resolve_runner` `NotImplementedError` -> Failure at `trajectory_hybrid.py:262`) is uncovered. Let me examine the key tests:

1. ``test_native_wasb_runner.py:502`` - ``test_resolve_runner_native_refused_for_tracknet`` - tests that ``_resolve_runner`` RAISES `NotImplementedError`, but NOT that ``process_video`` converts it to a Failure.
2. ``test_worker.py:860`` - mentions native/`NotImplementedError` returning rc 2.

Let me look at these two tests closely to determine whether they actually assert the ``process_video`` -> Failure conversion for the native path.

12. user (2026-07-06T18:05:38.157Z)

```

500
501
502     def test_resolve_runner_native_refused_for_tracknet(monkeypatch):
503         monkeypatch.delenv("RALLY_DETECTOR_IMPL", raising=False)
504         from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
505
506         with pytest.raises(NotImplementedError):
507             TrackNetHybridSegmenter(None, {}). _resolve_runner({"detector_impl": "native"})
508

```

13. user (2026-07-06T18:05:38.717Z)

```

855
856
857     def test_cmd_train_detector_native_on_tracknet_fails_clean_not_traceback(
858         tmp_path, monkeypatch
859     ):
860         """detector_impl=native for a family with no native runner must return rc 2 (clean),
not
861         propagate a raw NotImplementedError traceback out of the worker."""
862         monkeypatch.setenv("RALLY_DETECTOR_IMPL", "native")
863         corpus = tmp_path / "corpus"
864         labels_dir = corpus / "labels"
865         (labels_dir).mkdir(parents=True)
866         (corpus / "videos").mkdir(parents=True)
867         header = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
868         (labels_dir / "v1.rallies.csv").write_text(header + "1,2.0,5.0,winner,badminton,\n")
869         (labels_dir / "v2.rallies.csv").write_text(header + "1,3.0,6.0,winner,badminton,\n")
870         cfg = tmp_path / "config.json"
871         cfg.write_text("{}")
872         rc = wmain.main(

```

```

873         [
874             "train",
875             "--corpus-uri",
876             str(corpus),
877             "--out-uri",
878             str(tmp_path / "out"),
879             "--version",
880             "0.0.3",
881             "--config",
882             str(cfg),
883             "--scratch",
884             str(tmp_path / "s"),
885             "--trajectory-source",
886             "detector",
887             "--segmenter",
888             "tracknet_hybrid",
889         ]
890     )
891     assert (
892         rc == 2
893     ) # graceful: _resolve_train_detector caught NotImplementedError, returned None
894
895
896     def test_probe_video_fps_width_returns_none_on_unreadable(tmp_path):
897         """The strict ffprobe probe must return None (caller skips) for a non-video file,
898         never guess."""
899         bad = tmp_path / "not_a_video.mp4"
900         bad.write_bytes(b"NOTAVIDEO")

```

14. assistant (2026-07-06T18:05:48.437Z)

Now I've confirmed the key facts:

- ``test_resolve_runner_native_refused_for_tracknet`` (line 502) tests only that ``_resolve_runner`` **raises** `NotImplementedError` ? it directly calls ``_resolve_runner``, NOT ``process_video``. It does NOT assert the `trajectory_hybrid.py:262` conversion to ``Failure``.
- ``test_cmd_train_detector_native_on_tracknet_fails_clean_not_traceback`` (line 857) is about the **train** command (``_resolve_train_detector``), not ``process_video``'s inference path. Not relevant to the F1 conversion.

So the native-not-supported path in ``process_video`` (which returns ``Failure``) is indeed not covered by any test asserting the Failure. Neither is invalid-duration, unknown-provider, or unknown-sport.

Let me do a final confirmation that no test in the whole suite asserts a ``Failure`` for these three remaining paths (invalid duration, unknown provider, unknown sport) via `process_video`.

15. user (2026-07-06T18:05:50.387Z)

```

Found 6 files
tests\test_yolo_hybrid.py
tests\test_trajectory_hybrid_equivalence.py
tests\test_tracknet_hybrid.py
tests\test_motion_segmenter.py
tests\test_gemini.py
tests\test_base.py

```

16. assistant (2026-07-06T18:06:02.827Z)

The ``Failure`` assertions in `tracknet/yolo/equivalence` tests are exactly the ones I've already reviewed: no-api-key, healthcheck, no-trajectory, and all-errored-handoff (F3). The other test files (`motion_segmenter`, `gemini`, `base`) are different segmenters, not the `tracknet/yolo` pre-detection paths in question.

Let me verify the diff line references cited in the finding are accurate (line ~262 for `_resolve_runner`, ~236 for duration) by confirming the converted paths exist as described. The diff already shows all five conversions clearly:

- Unknown sport KeyError -> ``Failure(str(e))`` (diff line 49)
- Missing key -> ``Failure(msg, is_recoverable=True)`` (diff line 68) ? **tested**
- Unknown provider KeyError -> ``Failure(str(e))`` (diff line 76)
- Invalid duration -> ``Failure(...)`` (diff line 83-85)
- `_resolve_runner` NotImplementedError -> ``Failure(...)`` (diff line 98)

And yolo: `_resolve_provider`` bad sport (line 247), missing key (263, **tested**), unknown provider (272), and `process_video` invalid duration (399-401).

The finding is accurate. Of the F1/F2 pre-detection conversions, only the missing-API-key path has a test asserting ``Failure``. The other four converted paths (unknown sport, unknown provider, invalid duration, native-not-supported) have no test that exercises ``process_video`` and asserts a ``Failure`` return. A regression reverting any of them to ``return []``/``return None`` would pass the full suite.

This is a genuine test-coverage gap. Severity is appropriately ``low`` ? it's a missing-test / convention-tests observation, not a bug in shipped code. The code changes themselves are correct; the gap is that the safety net doesn't lock all the converted paths.

17. user (2026-07-06T18:06:12.010Z)

Structured output provided successfully